Adaptive Congestion-Control Simulator and Performance Analyzer Project Report

1. Abstract

Congestion control is an important function of computer networks that helps regulate the rate at which data is transmitted to avoid overloading the network. Different congestion-control algorithms respond to packet loss and changing network conditions in different ways, affecting throughput, congestion-window size, and recovery behavior.

The Adaptive Congestion-Control Simulator and Performance Analyzer is a Python-based application developed using Streamlit. It simulates and compares three congestion-control approaches: TCP Tahoe, TCP Reno, and Additive Increase Multiplicative Decrease (AIMD). Users can configure simulation parameters, examine congestion-window behavior, test different network conditions, compare algorithms, analyze performance metrics, and export results in CSV and JSON formats.

The project provides an interactive way to understand congestion-control behavior without requiring a physical network or packet-level TCP implementation. It uses a simplified, round-based simulation model and is intended for educational and comparative analysis.

2. Problem Statement

Understanding congestion-control algorithms requires examining how a sender adjusts its transmission behavior when network conditions change. Studying the algorithms theoretically does not always make their differences easy to visualize or compare.

A lightweight simulation tool is therefore needed to demonstrate the behavior of multiple congestion-control algorithms under configurable network conditions and present their performance through visualizations and quantitative metrics.

3. Objectives

- Implement simplified models of TCP Tahoe, TCP Reno, and AIMD.
- Simulate congestion-window changes over multiple rounds.
- Allow users to configure parameters such as packet-loss probability, RTT, number of rounds, initial congestion window, initial slow-start threshold, and random seed.
- Examine algorithm behavior under different network conditions, including ideal and high-loss scenarios.
- Compare algorithm performance using common metrics.
- Visualize simulation results through charts and tables.
- Export simulation data and results in CSV and JSON formats.
- Validate important simulation, algorithm, metric, and export behaviors through automated checks.

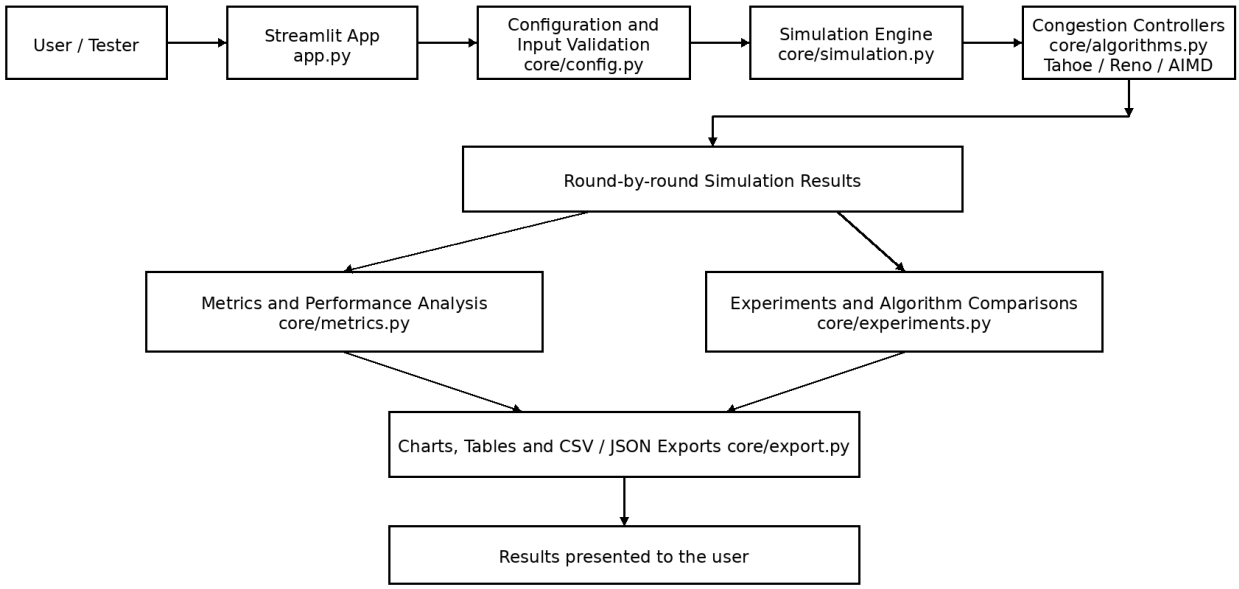
4. Technologies Used

- Python: Core programming language.
- Streamlit: Interactive user interface and dashboard.
- Pandas: Tabular simulation data and metric calculations.
- Plotly: Interactive charts and comparative visualizations.
- JSON and CSV: Data export formats.
- Python standard library: Configuration validation, file handling, and random-seed handling.

5. System Architecture

5.1 Architecture diagram

System Architecture of the Application



5.2 Module responsibilities

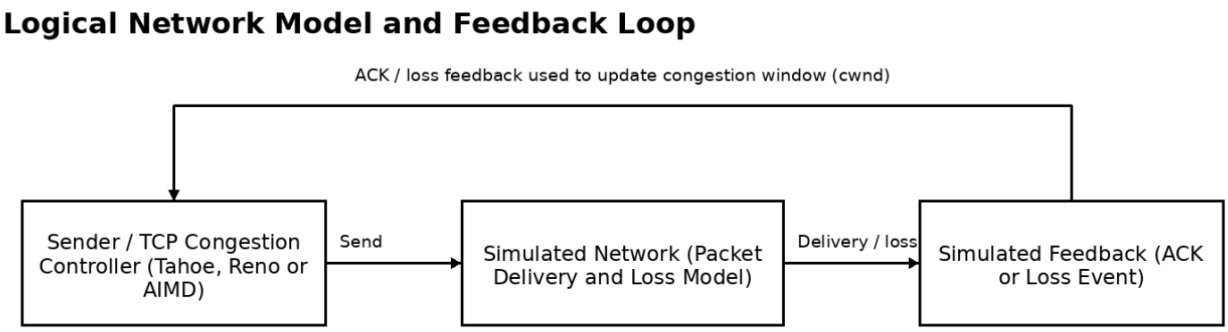
- app.py: Provides the Streamlit interface, configuration controls, navigation, charts, tables, and validation interface.
- core/config.py: Stores simulation parameters and validates their values.
- core/algorithms.py: Contains the congestion-control logic for Tahoe, Reno, and AIMD.
- core/simulation.py: Runs the round-based simulation and collects results.
- core/metrics.py: Calculates performance metrics from simulation data.
- core/experiments.py: Supports predefined network conditions and algorithm comparisons.
- core/export.py: Converts results into downloadable CSV and JSON data and supports saving files.

6. Network Model and Topology

This project uses a logical simulation model, rather than an actual network topology consisting of physical or virtual hosts, routers, and links.

The model represents a sender applying a congestion-control algorithm. Each simulation round represents one round-trip time (RTT). The simulation generates packet-delivery and packet-loss outcomes according to the configured loss probability. The selected algorithm uses the resulting feedback to update its congestion window.

Conceptually, the model can be represented as:



The sender, network, and feedback are logical components of the simulation. They do not exchange real packets over a network interface.

7. Protocol and Feedback Model

The application models congestion-control feedback at a simplified level. It records the number of packets sent, acknowledged, and lost in each round, along with congestion-window values and whether a congestion event occurred.

The simulation uses a heuristic to classify loss feedback:

- A round with at least four packets sent and exactly one packet lost is classified as a triple-duplicate-ACK-style event.
- Other loss cases are classified as timeout-style events.

This classification is an approximation used to demonstrate different algorithm responses. The simulator does not implement a complete TCP acknowledgment stream, retransmission timer, sequence-number system, or real packet exchange.

The round-by-round results form the simulation's internal record format. They include fields such as the round number, congestion window before and after the round, packets sent, packets lost, packets acknowledged, and loss-event information.

8. Congestion-Control Algorithms

8.1 TCP Tahoe

TCP Tahoe responds to congestion by reducing the congestion window to a small initial value, typically one segment in this model, and adjusting the slow-start threshold. It then resumes growth according to the model's congestion-control rules.

This behavior illustrates a conservative response to detected congestion.

8.2 TCP Reno

TCP Reno distinguishes between a triple-duplicate-ACK-style event and a timeout-style event in the simplified model.

- On a triple-duplicate-ACK-style event, Reno reduces its congestion window and models fast recovery.
- On a timeout-style event, Reno resets the congestion window more aggressively.

The distinction demonstrates how different types of loss feedback can produce different responses. It is an educational approximation of Reno rather than a complete implementation of the TCP Reno state machine.

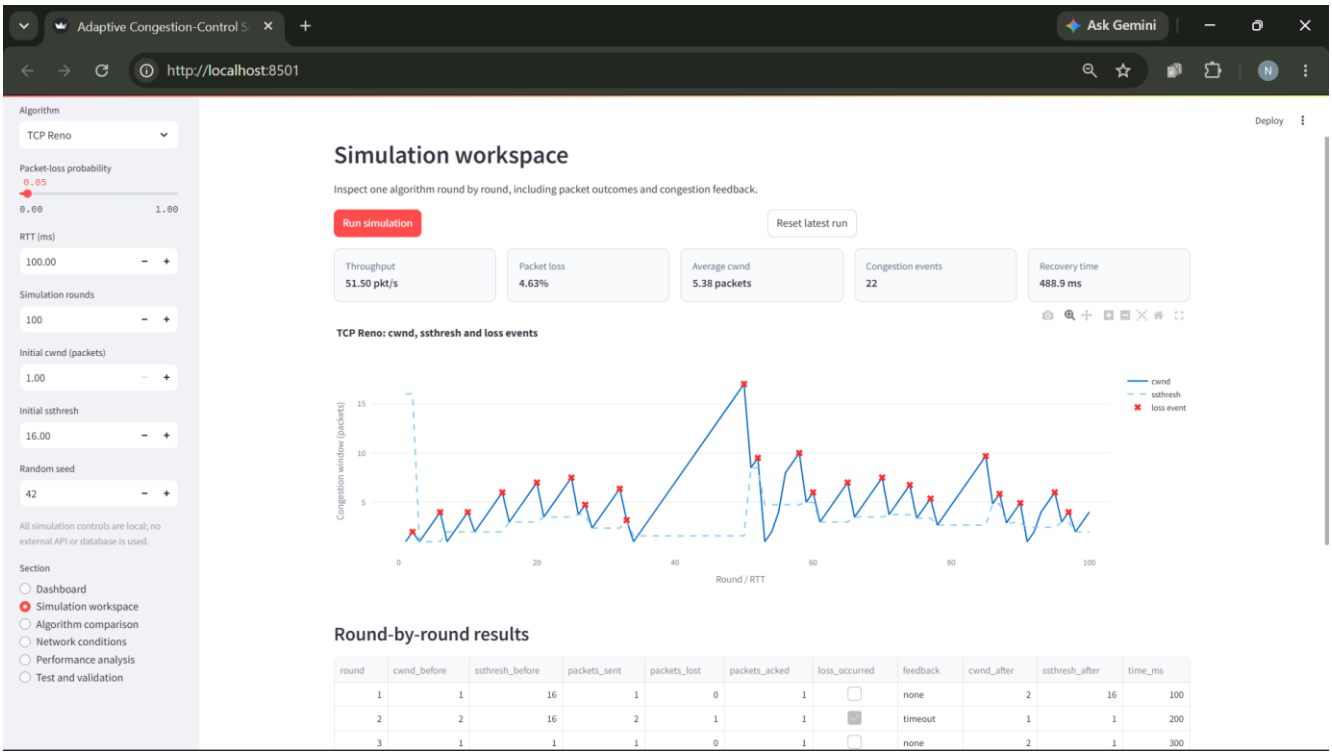
8.3 Additive Increase Multiplicative Decrease (AIMD)

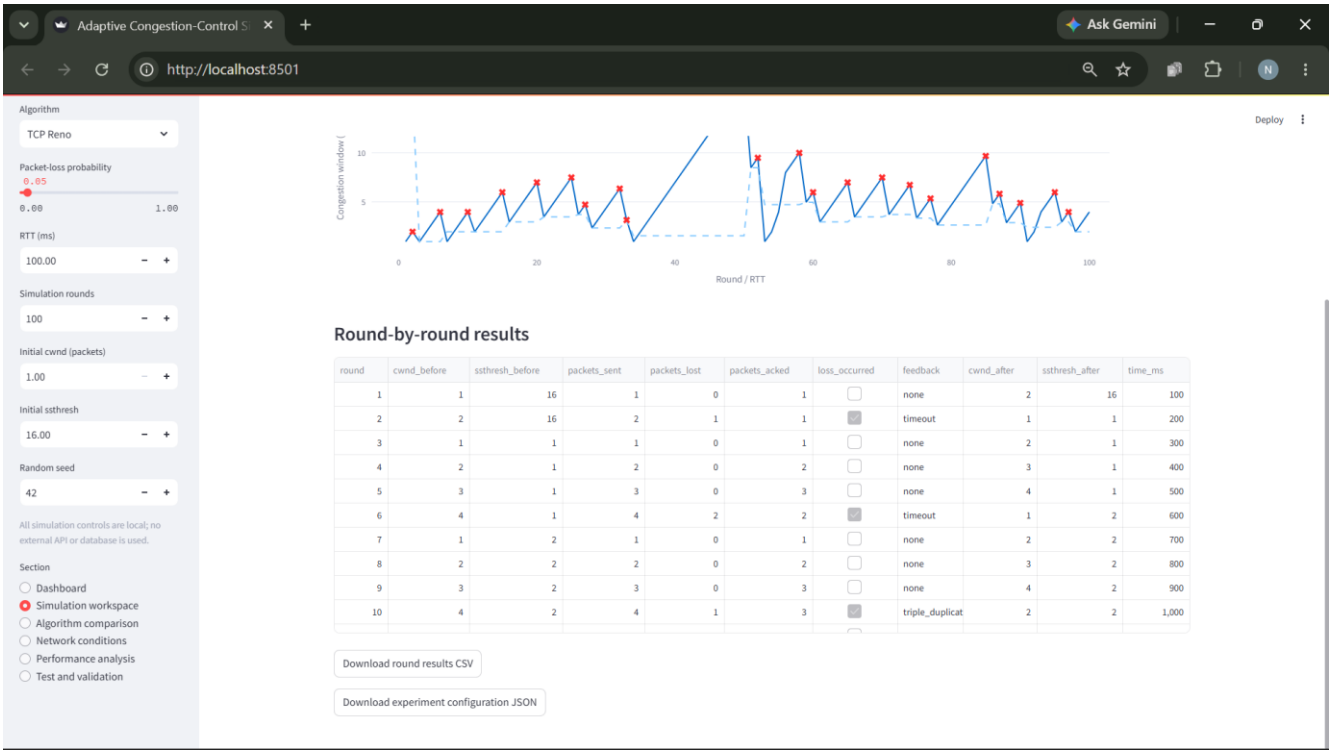
AIMD increases the congestion window gradually when no congestion is detected and multiplicatively reduces it when congestion occurs. In this project, the reduction is modeled by halving the congestion window.

AIMD serves as a baseline for examining gradual growth and congestion response.

9. Simulation Workflow

1. The user selects an algorithm and configures the simulation parameters.
2. The application validates the configuration.
3. The simulation initializes the algorithm's congestion-control state.
4. The simulation processes the configured number of rounds.
5. Each round generates packet-delivery or packet-loss outcomes.
6. The selected algorithm updates its congestion window based on the simulated feedback.
7. Round-by-round results are stored in a tabular structure.
8. Performance metrics are calculated from the results.
9. The application displays charts, tables, comparisons, and downloadable results.





A configurable random seed supports repeatable experiments when the same settings and implementation are used.

10. Performance Metrics

10.1 Throughput

Throughput is calculated as the number of acknowledged packets divided by the total simulated time.

Throughput = Total Acknowledged Packets / (Number of Rounds × RTT in Seconds)

The result is expressed in packets per second. It is a simplified application-level measure, not a measurement of real network bandwidth.

10.2 Packet-Loss Rate

Packet-Loss Rate = Total Packets Lost / Total Packets Sent

The ratio can be expressed as a percentage by multiplying it by 100.

10.3 Average Congestion Window

The application calculates the mean of the recorded congestion-window values before each round. This provides a summary of the sender's congestion-window behavior throughout the simulation.

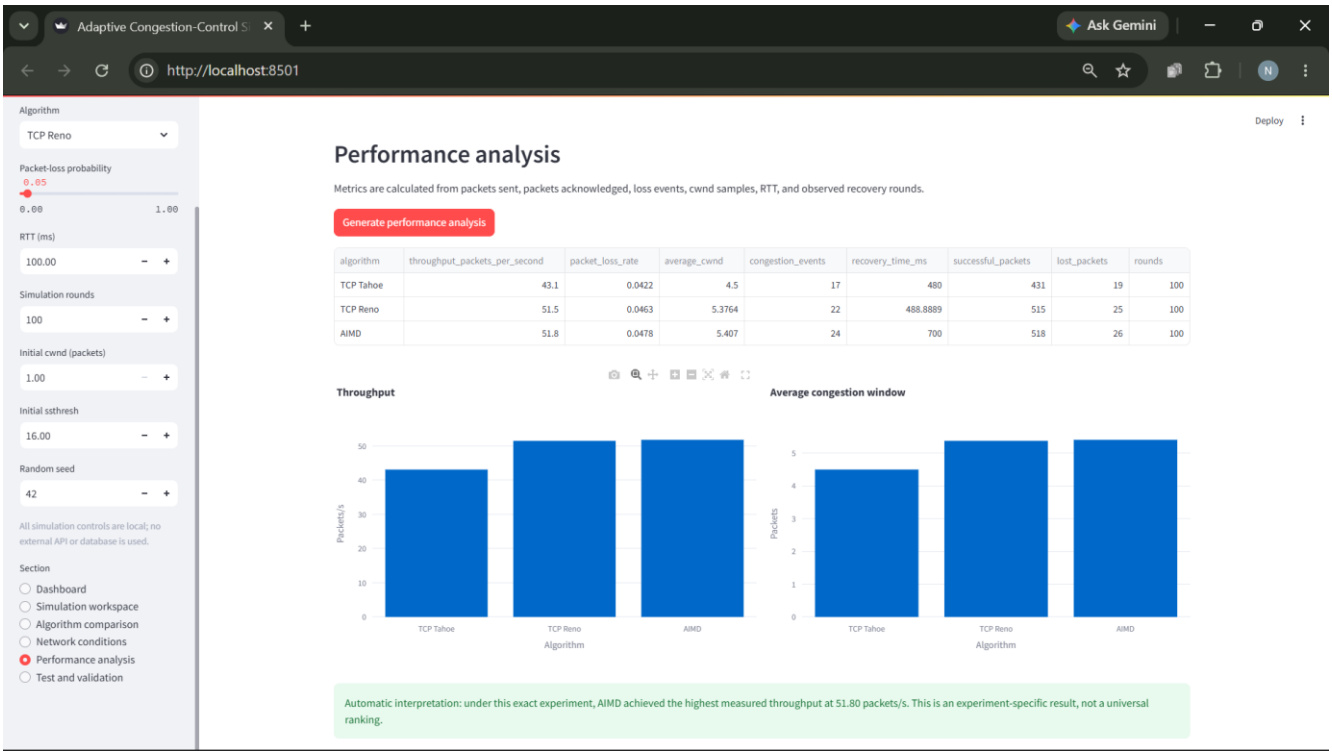
10.4 Congestion Events

The congestion-event count is the number of rounds marked as having a loss-related congestion event.

10.5 Recovery Time

Recovery time is estimated from the first subsequent round in which the congestion window after the round reaches or exceeds the congestion-window value recorded before the loss round. The metric is averaged across loss events that satisfy this condition. Events without a qualifying recovery are excluded; if none qualify, the result is unavailable.

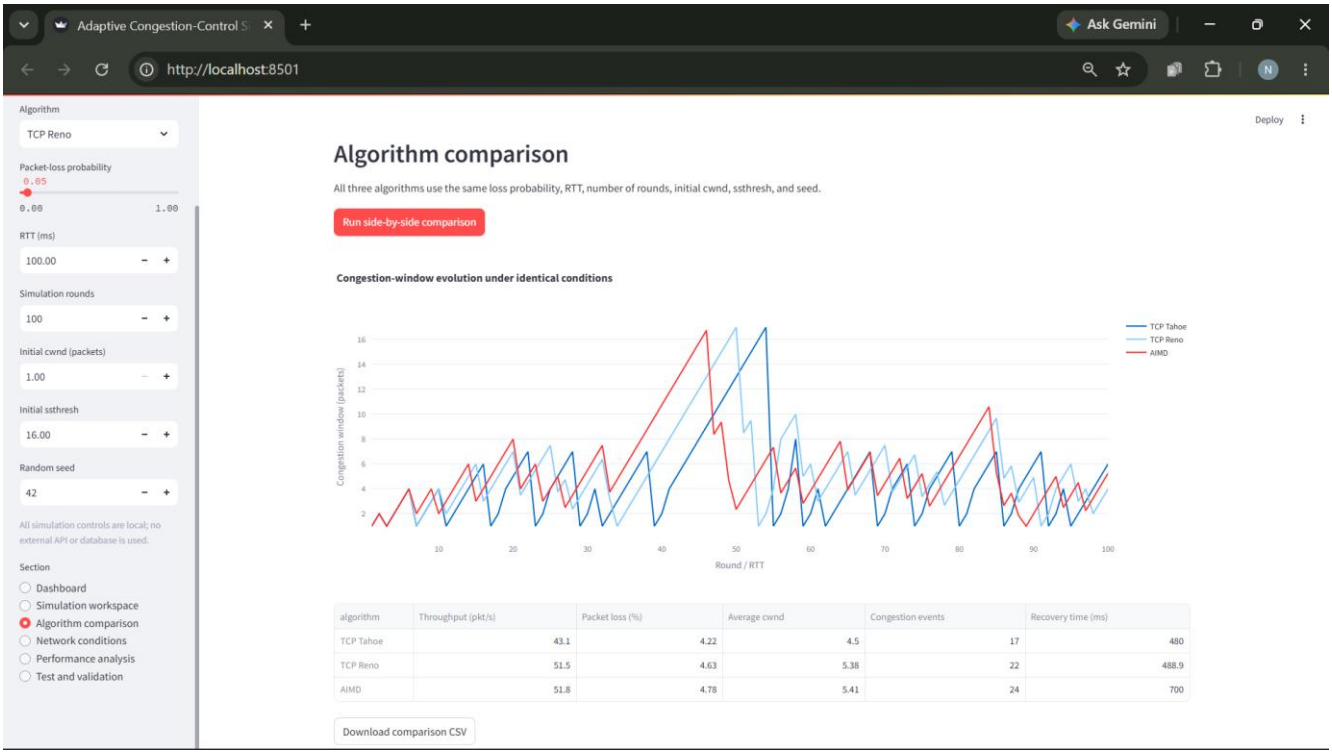
This is a simplified recovery indicator, not a direct measurement of real TCP recovery time.

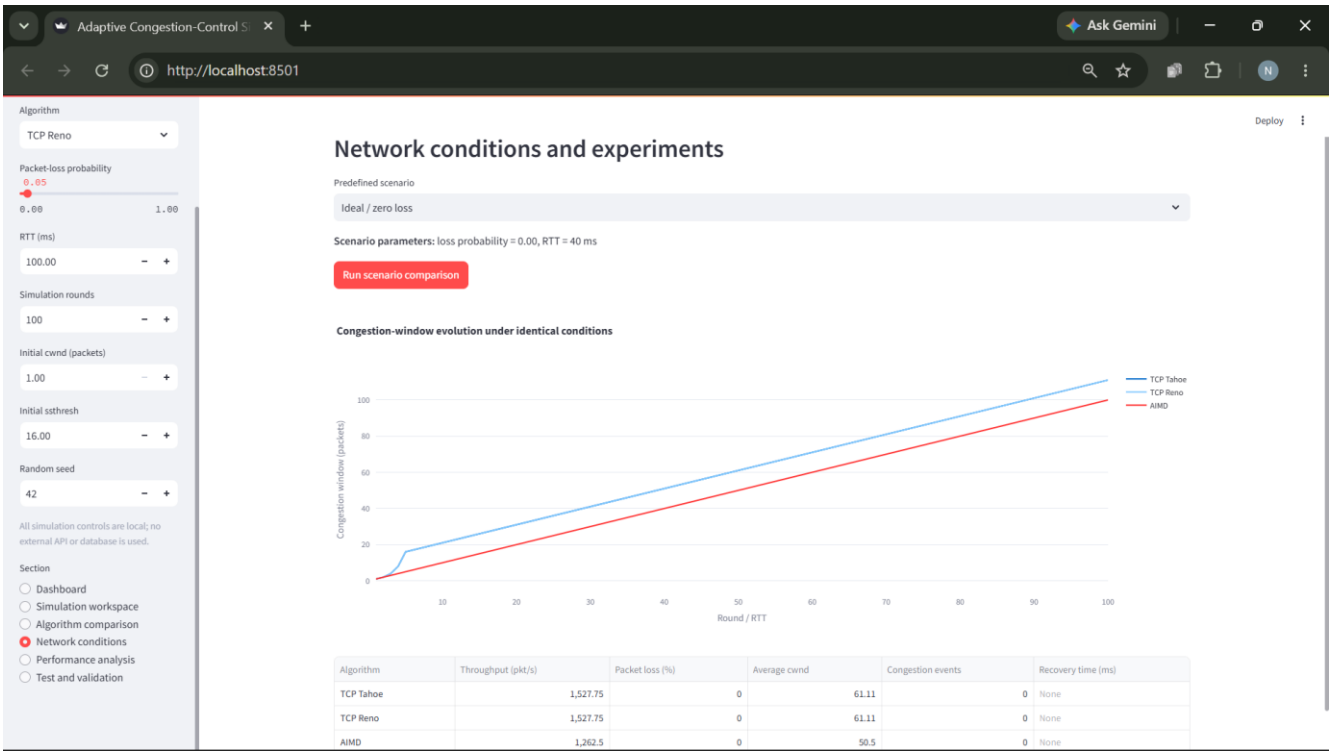


11. Implementation Features

The application provides:

- A dashboard for viewing simulation information.
- A simulation workspace for configuring and running individual simulations.
- An algorithm-comparison section.
- A network-conditions section with predefined scenarios.
- A performance-analysis section for examining calculated metrics.
- A validation section for running built-in checks.
- Charts and tables showing simulation results.
- CSV and JSON export functionality.
- Input validation for invalid simulation configurations.
- Seed-based repeatability for comparable experiments.





12. Testing and Validation

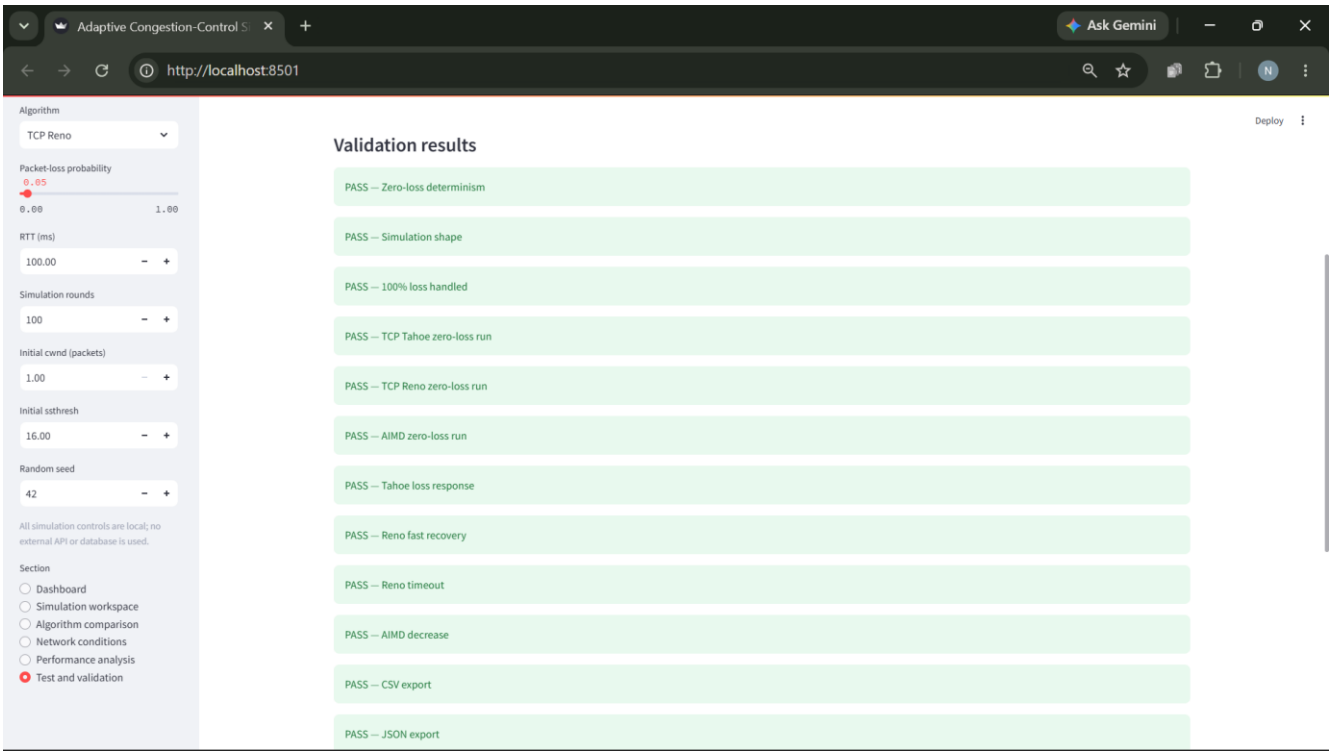
The built-in validation suite was run successfully, with all 20 checks passing.

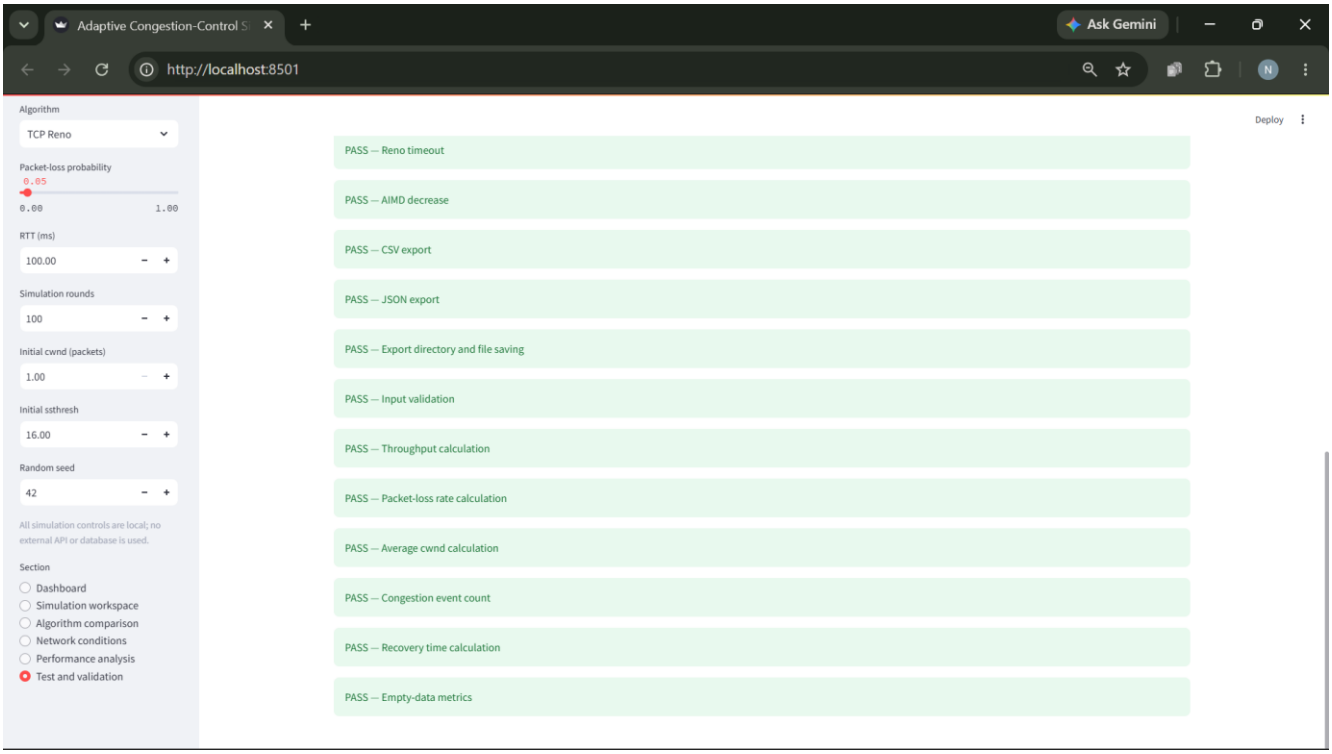
The checks cover:

Area	Validation
Repeatability	Zero-loss determinism
Simulation output	Expected simulation shape

Extreme conditions	Handling of 100% packet loss
Algorithm behavior	Zero-loss runs for Tahoe, Reno, and AIMD
Tahoe	Loss response
Reno	Fast-recovery-style response and timeout response
AIMD	Congestion-window decrease
Export conversion	CSV and JSON generation
File handling	Output-directory creation and file saving
Configuration	Input validation
Metrics	Throughput, packet-loss rate, average congestion window, congestion-event count, recovery time, and empty-data handling

The Streamlit interface was also manually checked. The simulation workspace, algorithm comparison, network-conditions section, performance analysis, and CSV/JSON downloads worked during the manual walkthrough.





12.1 Test-case summary

- Ideal conditions: Zero-loss simulations were checked for determinism and algorithm execution.
- Failure conditions: A 100% packet-loss scenario was checked to ensure the simulation handles extreme loss.
- Algorithm behavior: Loss responses, Reno's two modeled loss-response paths, and AIMD's decrease behavior were checked.
- Metrics and exports: Calculations, serialization, and file saving were checked.

The built-in checks verify selected properties of the implementation; they do not prove that the simulator reproduces every aspect of real TCP behavior.

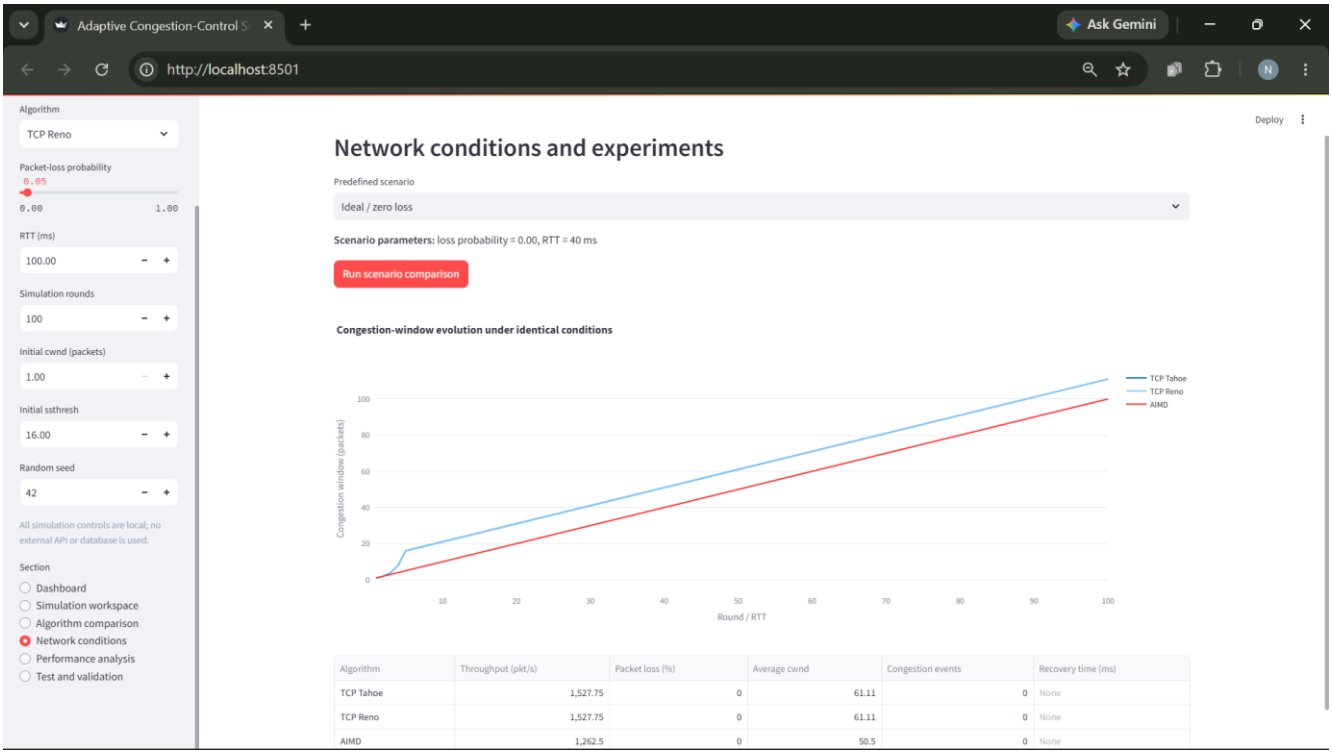
13. Results and Discussion

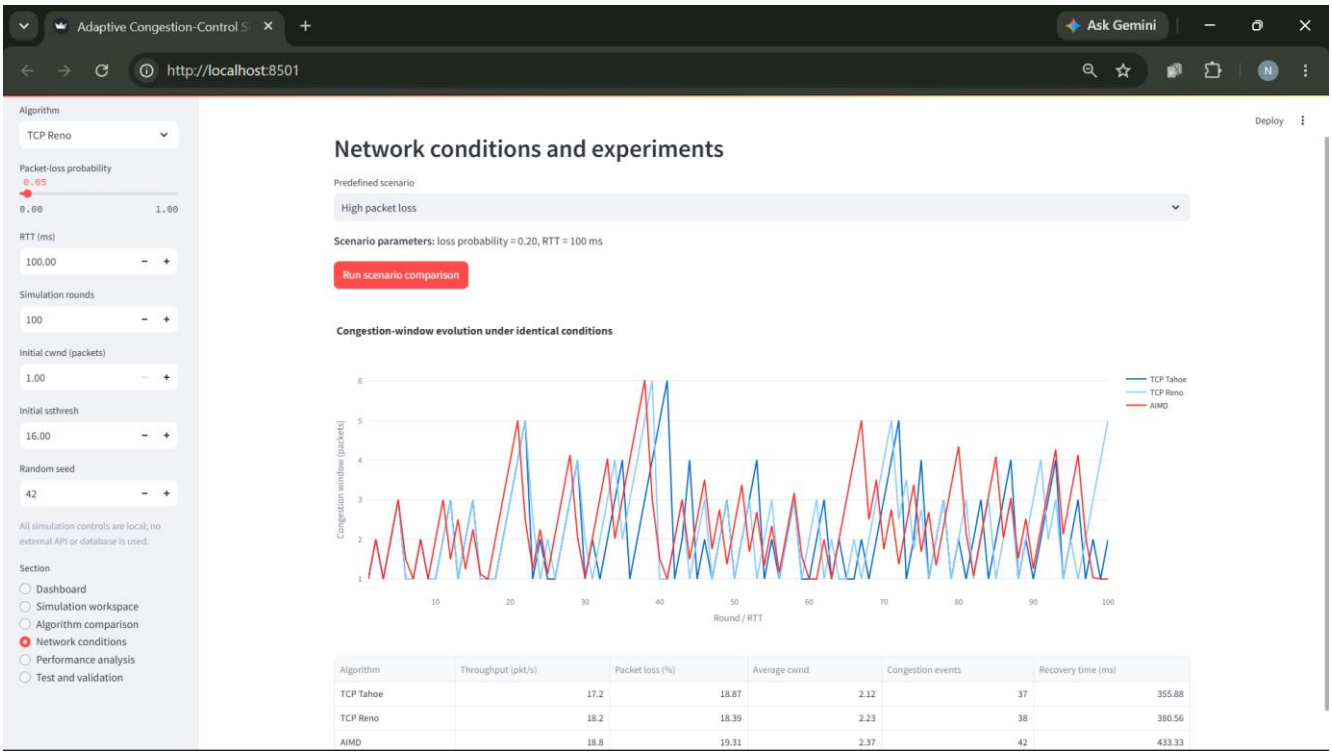
The simulator enables comparisons across algorithms and network conditions using configurable parameters and common performance metrics.

Under ideal conditions, Tahoe and Reno may produce overlapping congestion-window curves because their modeled window-growth behavior can be identical when no packet loss occurs.

Increasing packet loss can reduce throughput and trigger additional congestion responses. The relative performance of Tahoe, Reno, and AIMD depends on the selected conditions and the simplifications in the simulation model.

The results represent simulated behavior rather than measurements from a real network.





14. Limitations

- It uses a round-based simulation rather than real TCP packet transmission.
- Packet loss is generated probabilistically.
- Loss-event classification uses a heuristic rather than actual duplicate ACKs and retransmission timers.
- It does not model real network queues, routers, bandwidth bottlenecks, propagation delay variation, or competing traffic.

- Throughput and recovery time are calculated from simplified simulation data.
- The simulator does not provide Wireshark packet-level verification because it does not create real TCP traffic.

15. Conclusion

The Adaptive Congestion-Control Simulator and Performance Analyzer provides an interactive environment for studying simplified TCP Tahoe, TCP Reno, and AIMD congestion-control behavior. It combines configurable simulations, performance metrics, algorithm comparisons, visualizations, and data exports in a modular Python application.

The built-in validation suite and manual interface walkthrough confirm that the tested application features operate as intended. Although it does not replace real network emulation or packet-level analysis, the project provides a practical educational tool for exploring congestion-control concepts and comparing algorithm behavior under different simulated conditions.

16. Team Contributions

- Project planning and problem definition - **Neel Rede**
- Congestion-control algorithm implementation - **Arya Kale**
- Simulation engine and configuration validation - **Parth Agrawal**
- Streamlit interface and visualizations - **Parth Agrawal**
- Network scenarios and algorithm comparisons - **Saumya Shah**
- Performance metric calculations - **Saumya Shah**
- Testing and validation - **Neel Rede**
- CSV/JSON export functionality - **Neel Rede**
- README and demonstration - **Neel Rede**
- Architecture and technical documentation - **Arya Kale**

Github: https://github.com/Programmerboi2006/Adaptive-Congestion-Control-Simulator-Group-2